Stokvel Management Platform

UML 4+1 Diagram Package — Sprint 3

RateLimitRefugees • May 2026

The 4+1 Architectural View Model

The 4+1 model describes a software architecture using five concurrent views, each addressing a different set of concerns for different stakeholders.

1	2	3	4	+1
Logical View	**Development View**	**Process View**	**Physical View**	**Scenario View**
Use Case Diagram	**Component Diagram**	**Sequence Diagram**	**Deployment Diagram**	**ERD**
Functional requirements actor / use-case map	Static code organisation application layers	Runtime behaviour login + SARB flow	Infrastructure topology browser → Supabase Cloud	Data model + relationships across all six tables

Page index

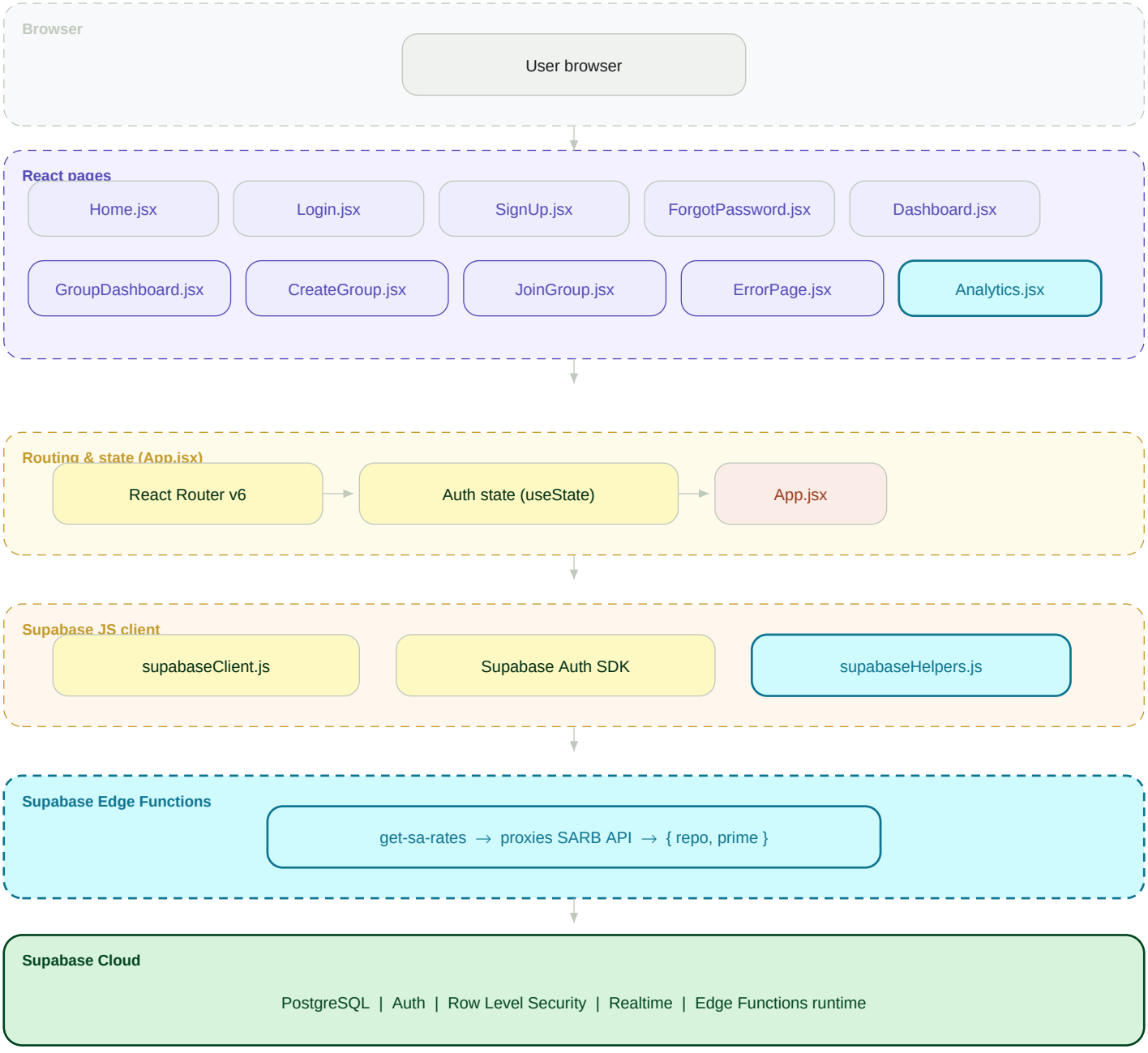
2. Logical View — Use Case Diagram

Shows who (actors) can do what (use cases). Covers all functional requirements visible to end-users.

Teal = Sprint 3 additions.

3. Development View — Component Diagram

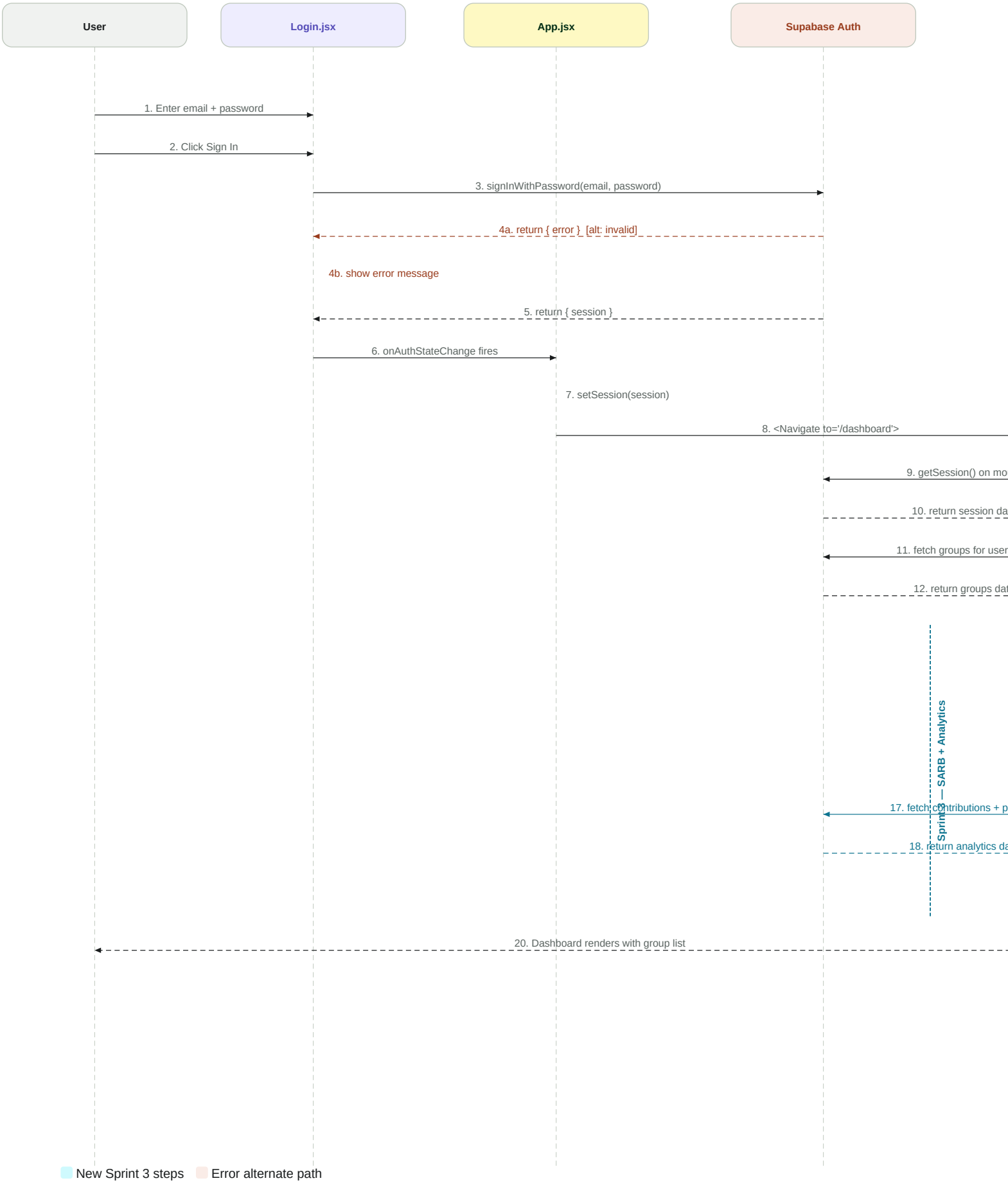
Static software organisation: browser → React pages → Router → Supabase client → Edge Functions → Supabase Cloud.



React pages Routing / client New Sprint 3

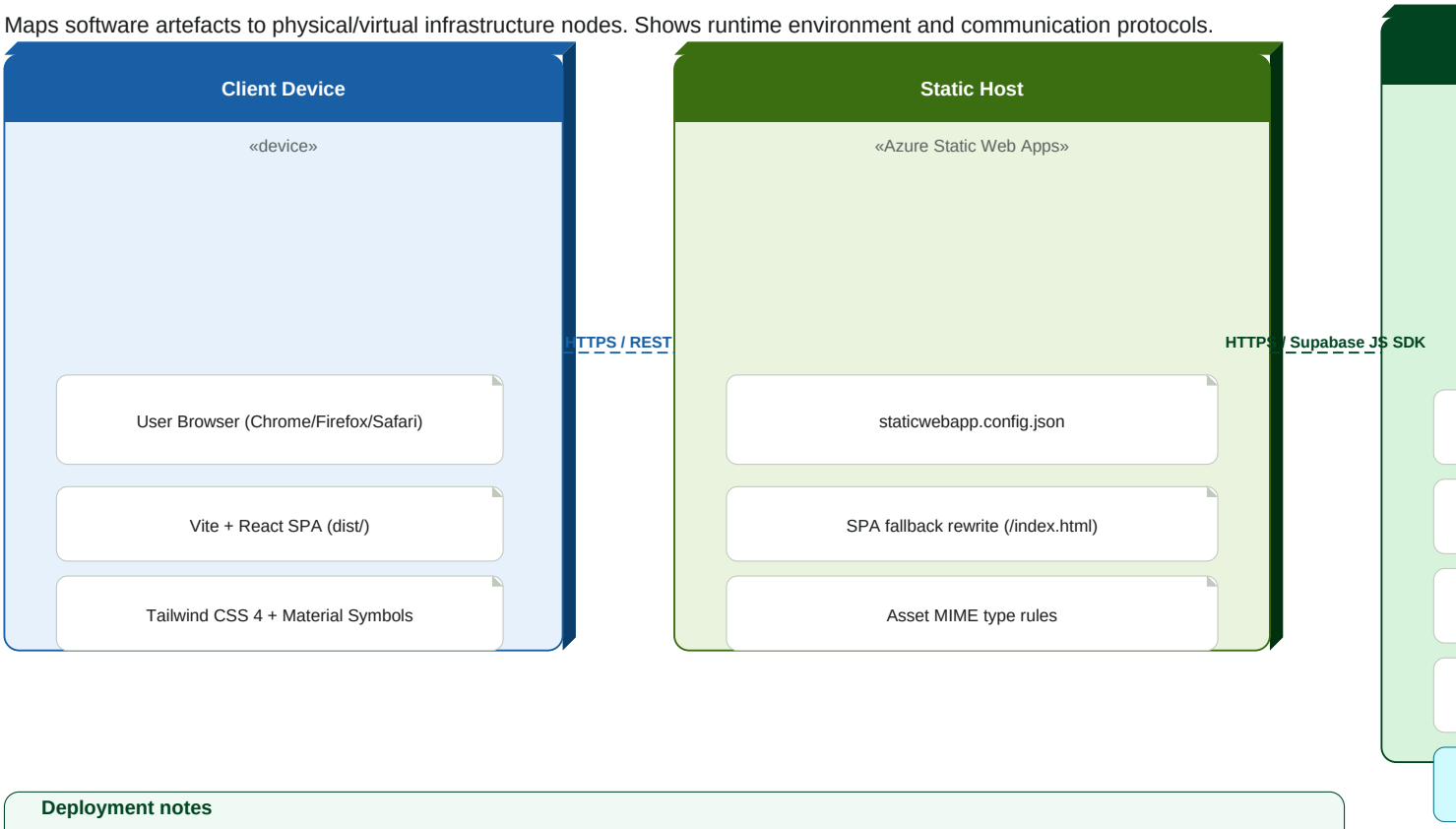
4. Process View — Sequence Diagram

Runtime behaviour: login propagation + SARB live-rates fetch + savings projections + analytics data load.



5. Physical View — Deployment Diagram

Maps software artefacts to physical/virtual infrastructure nodes. Shows runtime environment and communication protocols.



Deployment notes

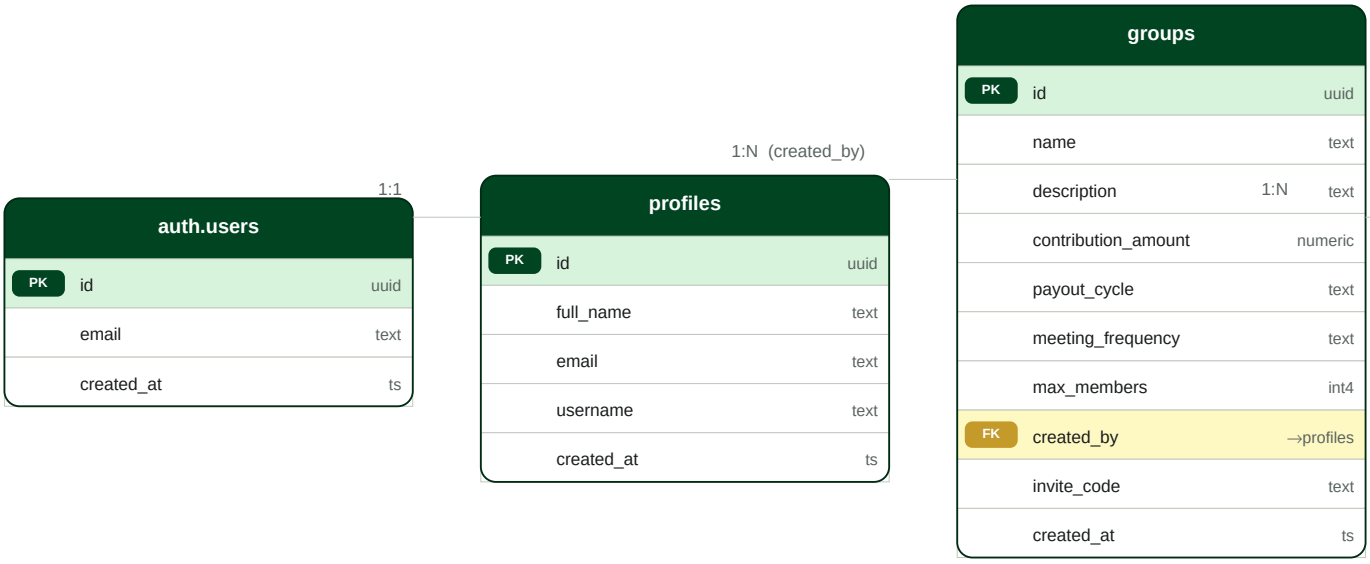
- Build tool: Vite 8 (React 19, Rolldown bundler) | CSS: Tailwind 4 with @tailwindcss/vite plugin
- Environment secrets (VITE_SUPABASE_URL, VITE_SUPABASE_KEY) stored in Netlify/Azure env vars — never committed to Git
- Supabase Edge Functions run on Deno; deployed via supabase functions deploy get-sa-rates

Client tier Hosting tier Supabase Cloud External API New Sprint 3

6. Scenario View (+1) — Entity Relationship Diagram

Database schema across seven tables. auth.users managed by Supabase Auth. Green = PK, Yellow = FK, Teal = new Sprint 3 field.

Part 1 of 2 — auth.users, profiles, groups, payouts



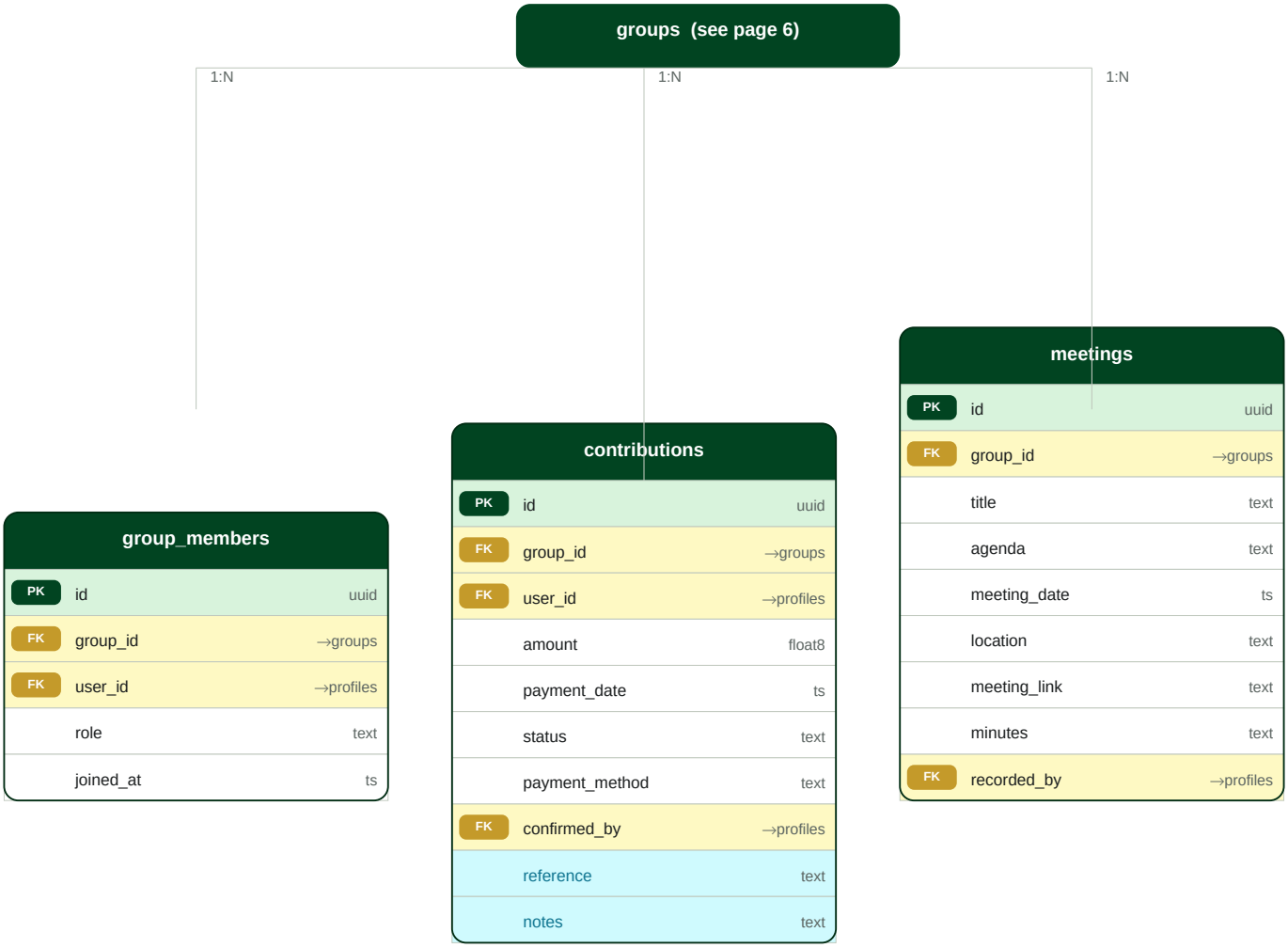
Design decisions & RLS notes

auth.users — Managed entirely by Supabase Auth; application code never writes to this table directly.
profiles — Extended with trigger handle_new_user() that auto-inserts a row on auth.users INSERT.
groups — Core entity. All child tables (group_members, contributions, payouts, meetings) carry group_id FK.
payouts — initiated_by FK added Sprint 3 to support treasurer audit trail. status ∈ {pending, completed, cancelled}.
RLS — Every table has RLS enabled. Helper functions get_user_group_ids() and is_group_admin() avoid recursive policy joins. Members can only read rows where they are a group_member.
Indexes — group_id columns indexed on all child tables; user_id indexed on group_members and contributions.

PK — primary key FK — foreign key New field Sprint 3

6. Scenario View (+1) — ERD (continued)

Part 2 of 2 — group_members, contributions, meetings; all three reference the groups table.



RLS policy summary — these three tables

```
group_members   —  SELECT: member can read rows for groups they belong to (via get_user_group_ids helper).
                   INSERT: user_id must equal auth.uid() — users can only add themselves.
                   UPDATE: only group admin (is_group_admin helper) can change roles.
contributions   —  SELECT: own rows OR any row in the user's groups (treasurer visibility).
                   INSERT/UPDATE: user_id = auth.uid(). Treasurer/admin can confirm or flag.
                   reference and notes fields (Sprint 3) visible to all group members.
meetings        —  SELECT: all members of the group.
                   INSERT/UPDATE/DELETE: role ∈ {admin, treasurer} only.
```

PK — primary key FK — foreign key New field Sprint 3